
LLD – DAY 10

Singleton Design Pattern

◇ Introduction

Singleton ek aisa design pattern hai jisme:

Class apne aap ko restrict karti hai ki sirf ek hi object (instance) ban sake. Agar koi dubara object maange, toh **wahi same instance return hota hai.**

🔗 Hinglish me:

"Object ek hi banta hai, baar-baar wahi use hota hai."

◇ Why Singleton? (Need)

- Resource expensive objects (DB connection, logger)
 - Global configuration
 - Shared state manage karna
 - Centralized access point chahiye
-

◇ Understanding Object Creation (Under the Hood)

Jab hum likhte hain:

```
A* a = new A();
```

Internally kya hota hai:

1. **Heap memory** me class **A** ka object banta hai
 2. **Constructor call** hota hai
 3. Object ka **address heap me hota hai**
 4. Pointer **a** **stack me store hota hai** aur heap address point karta hai
-

◇ Memory Types

🧠 Stack Memory

- Primitive types (int, char, bool)
- Local variables
- Fast but limited

🧠 Heap Memory

- Objects (using **new**)
- Dynamic allocation
- Manual management

◇ Problem: Multiple Object Creation (Without Singleton)

✗ No Singleton Example

```
class NoSingleton {
public:
    NoSingleton() {
        cout << "New Object created" << endl;
    }
};

int main() {
    NoSingleton* s1 = new NoSingleton();
    NoSingleton* s2 = new NoSingleton();

    cout << (s1 == s2) << endl; // false
}
```

🔗 Issue:

- Har **new** pe naya object
- Memory waste
- No control

◇ How to Stop Multiple Object Creation

☑ Core Rules of Singleton

1. Constructor ko **private** banao
2. Ek **static instance variable**
3. Ek **static getInstance() method**
4. Object creation control sirf class ke paas

◇ Method 1: Basic Singleton (Lazy Initialization)

```
class Singleton {
private:
    static Singleton* instance;

    Singleton() {
```

```
        cout << "Singleton Constructor called" << endl;
    }

public:
    static Singleton* getInstance() {
        if (instance == nullptr) {
            instance = new Singleton();
        }
        return instance;
    }
};

Singleton* Singleton::instance = nullptr;
```

✓ Pros

- Object tabhi banta hai jab zarurat ho

✗ Cons

- **Thread-safe nahi**
- Multi-threading me multiple instance ban sakte hain

◇ Problem with Multithreading

Agar 2 threads **same time** `getInstance()` call karein:

- Dono ko `instance == nullptr` dikhega
- Dono `new Singleton()` bana denge ✗

◇ Method 2: Thread-Safe Singleton (Using Mutex)

```
class Singleton {
private:
    static Singleton* instance;
    static mutex mtx;

    Singleton() {}

public:
    static Singleton* getInstance() {
        lock_guard<mutex> lock(mtx);
        if (instance == nullptr) {
            instance = new Singleton();
        }
        return instance;
    }
};
```

✓ Pros

- Thread-safe

✗ Cons

- Har call pe **lock lagta hai**
 - Performance expensive
-

◇ Method 3: Double Check Locking (Optimized)

```
class Singleton {
private:
    static Singleton* instance;
    static mutex mtx;

    Singleton() {}

public:
    static Singleton* getInstance() {
        if (instance == nullptr) {           // First check
            lock_guard<mutex> lock(mtx);
            if (instance == nullptr) {       // Second check
                instance = new Singleton();
            }
        }
        return instance;
    }
};
```

✓ Pros

- Thread-safe
 - Lock sirf pehli baar
 - Best practical approach
-

◇ Method 4: Eager Initialization

```
class Singleton {
private:
    static Singleton* instance;

    Singleton() {}

public:
    static Singleton* getInstance() {
        return instance;
    }
};
```

```
    }  
};  
  
Singleton* Singleton::instance = new Singleton();
```

✓ Pros

- Thread-safe by default
- Simple

✗ Cons

- Object banega **chahe use ho ya na ho**
- Memory waste ho sakta hai

🔗 Use when:

Tum sure ho object hamesha use hoga

◇ Overall Working of Singleton

1. Constructor private
2. Static instance variable
3. Static `getInstance()` method
4. Same object har baar return

◇ Text UML Diagram – Singleton

```
Singleton  
-----  
| - instance : Singleton |  
-----  
| - Singleton()          |  
| + getInstance()        |  
-----
```

◇ Real-World Use Cases

1. Logging System

- Ek hi logger poore app me

2. Database Connection

- Single DB connection pool

3. Configuration Manager

- App config ek jagah se load
-

◇ Where NOT to Use Singleton

1. ✗ Jab multiple instance chahiye
 - Online games
 - Multi-user session
 2. ✗ Testing difficult
 - Mocking hard
 - Unit test complex
 3. ✗ Hidden dependencies
 - Tight coupling
-

📝 Quick Revision (Exam Ready)

- Singleton = one instance only
 - Constructor private
 - Static instance + getInstance
 - Lazy vs Eager
 - Thread safety important
-

🎯 Interview One-Liners

- **Singleton ensures a class has only one instance**
 - **Global access point provided**
 - **Be careful in multithreaded environment**
 - **Avoid Singleton if flexibility is needed**
-